

TP1 – Device Driver de FreeRTOS

Objetivos

Diseñar, Implementar y Usar **Device Driver** de **FreeRTOS**

Estructura de datos que representen al dispositivo, incluyendo: **device_id**, **device_queue**, etc.

Funciones de Interfaz: **open()**, **release()**, **write()**, **read()**, **ioctl()**, etc.

Patrones de diseño: **Synchronous**, **Asynchronous**, **Latest Input Only**

Gestión del periférico: **Polling**, **Interrupt**, **DMA**

Acceso al periférico: **API STM32-F4 HAL**

Tareas del tipo: **Gatekeeper**

Almacenamiento de datos: **Queue**, **Queue with Input and Output Data Spooler** (allocation: **Static**, **Dynamic**)

Usar **Tareas** de **FreeRTOS**

Diseñar, Implementar, Crear, Eliminar **Tareas** (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias, parámetros, cambio de prioridades, procesamiento periódico, IDLE)

Usar **Comunicación** y **Sincronización** de **Tareas** e **Interrupciones** de **FreeRTOS**

Diseñar, Implementar, Comunicar, Sincronizar **Tareas** entre sí y **Tareas** con **Interrupciones**, mediante **Cola** (bloqueantes o no bloqueantes) o **Semáforo binario** y/o **contador** (bloqueantes o no bloqueantes)

Usar **Sección Crítica** de **FreeRTOS**

Diseñar, Implementar, Proteger **Sección Crítica**, mediante **taskENTER_CRITICAL()/taskEXIT_CRITICAL()**, **vTaskSuspendAll()/xTaskResumeAll()** y **Semáforo Mutex** (Take/Give)

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Editar/compilar/depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/actualizar **repositorio** por línea de **comando**

Generar/actualizar **archivos** del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/explicar **código fuente**

Actividades

[TP1-00 – Formalismos](#)

[TP1-01 – Device Driver I2C de FreeRTOS](#)

[TP1-02 – Device Driver UART de FreeRTOS](#)

[TP1-03 – Device Driver ADC de FreeRTOS](#)

Resolución en grupos de 3 alumnos, requiere **presentación** por parte de un **responsable** del grupo

TP1 – Actividad 00 – Formalismos

Objetivos

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/Explicar código fuente

Paso 01: Crear un nuevo **repositorio público** en **GitHub** para almacenar su **TP1**, pasos:

GitHub: Repositories => New =>

Repository name: **sotrii-tp1_Cohorte-Grupo**

Description: # **CESE - Sistemas Operativos de Tiempo Real II - Trabajo Práctico N°: 1 – Device Driver de FreeRTOS**

Initialize this repository with:

Tildar **Add a README file**

=> **Create repository**

Campus FIUBA Grado => **10. SOTR (II) > TP's - Enunciados > Clase 2 > SOTR_II - Guia TP1 - Device Driver**

➡ **Completar** [Registro de Entrega de TP](#) 🔔

➡ **Completar** [Encuesta de Bienvenida](#) 🔔

Cohorte-Grupo es: **2XCo202X-01** o **2XCo202X-02** ... etc. (el correspondiente al Curso y Grupo de cursada)

Paso 02: Invitar al **repositorio**, como **colaboradores** a los **docentes** de su curso, responsables de la aprobación de su **TP1**, pasos:

GitHub: Repositories => Settings => Collaborators => Tildar: **Add people** =>

en **Find people** => tipear: **JuanManuelCruz-FIUBA**

Tildar: **Add JuanManuelCruz-FIUBA**

GitHub: Repositories => Settings => Collaborators => Tildar: **Add people** =>

en **Find people** => tipear: **Cpatagon**

Tildar: **Add Cpatagon**

Una vez que los [docentes](#) de su curso, **hayan aceptado** la **invitación** a ser [colaboradores](#), podrá agregarlos como **Revisores**

PasPaso 03: Asentar los detalles de las **entregas** en archivos del tipo **README.md** ([Markdown](#), no docx o pdf u otros formatos), **editar** el archivo **README.md** y **modificar** su contenido, pasos:

GitHub: [README.me](#) => **Edit this file** => **borrar** su contenido => **Copiar y Pegar** lo siguiente:

```
# CESE - Sistemas Operativos de Tiempo Real II
## Trabajo Práctico N°: 1 - Device Driver de FreeRTOS
### Cohorte-Grupo: . . . . .
### Modelo de placa: . . . . .

### Responsable de la entrega
| N° SIU | Apellidos, Nombres | Fecha | Deadline |
| :----- | :----- | :-----: | :-----: |
| XXXXXX | YYYY, ZZZ | | Semana 04 |
```

Actualizar el archivo **README.md** con los datos de **Cohorte-Grupo**, **Modelo de placa**, del **Responsable de la entrega** (un alumno diferente para cada TP):

26Co2026 - Grupo_1, **NUCLEO-F446RE**, **N° SIU**, **Apellidos** y **Nombres**

TP1 – Actividad 01 – Device Driver I2C de FreeRTOS

Objetivos

Diseñar, Implementar y Usar **Device Driver** de **FreeRTOS**

Estructura de datos que representen al dispositivo, incluyendo: **device_id**, **device_queue**, etc.

Funciones de Interfaz: **open()**, **release()**, **write()**, **read()**, **ioctl()**, etc.

Patrones de diseño: **Synchronous**, **Asynchronous**, **Latest Input Only**

Gestión del periférico: **Polling**, **Interrupt**, **DMA**

Acceso al periférico: **API STM32-F4 HAL**

Tareas del tipo: **Gatekeeper**

Almacenamiento de datos: **Queue**, **Queue** with **Input and Output Data Spooler** (allocation: **Static**, **Dynamic**)

Usar **Tareas** de **FreeRTOS**

Diseñar, Implementar, Crear, Eliminar **Tareas** (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias, parámetros, cambio de prioridades, procesamiento periódico, IDLE)

Usar **Comunicación** y **Sincronización** de **Tareas** e **Interrupciones** de **FreeRTOS**

Diseñar, Implementar, Comunicar, Sincronizar **Tareas** entre sí y **Tareas** con **Interrupciones**, mediante **Cola** (bloqueantes o no bloqueantes) o **Semáforo binario** y/o **contador** (bloqueantes o no bloqueantes)

Usar **Sección Crítica** de **FreeRTOS**

Diseñar, Implementar, Proteger **Sección Crítica**, mediante **taskENTER_CRITICAL()/taskEXIT_CRITICAL()**, **vTaskSuspendAll()/xTaskResumeAll()** y **Semáforo Mutex** (Take/Give)

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/Explicar código fuente

Paso 01: Generar un nuevo **proyecto STM32**: **descargar** el archivo [sotrii-tp1_01-application.zip](#) e **importar** el **proyecto STM32** que contiene: **sotrii-tp1_01-application**.

Compilar el **proyecto STM32** (**STM32CubeIDE**) importado: **sotrii-tp1_01-application**.

Paso 02: Asentar los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **sotrii-tp1_01-application.md**.

Paso 03: Consultar a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

Ingresar a **Gemini** => <https://gemini.google.com/app>, en **Preguntar a Gemini** =>

Copiar y Pegar la siguiente línea de **texto**:

Analizar y explicar (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: [app.c](#), [app_it.c](#), [task_sender.c](#), [task_receiver.c](#), [task_i2c.c](#), [task_i2c_interface.c](#), [task_i2c_interface.h](#) y [freertos.c](#).

Agregar los siguientes archivos (click en  Agregar archivos => **Subir archivos**):

```
...\sotri_workspace\sotrii-tp1_01-application\app\src\app.c
...\sotri_workspace\sotrii-tp1_01-application\app\src\app_it.c
...\sotri_workspace\sotrii-tp1_01-application\app\src\task_sender.c
...\sotri_workspace\sotrii-tp1_01-application\app\src\task_receiver.c
...\sotri_workspace\sotrii-tp1_01-application\app\src\task_i2c.c
...\sotri_workspace\sotrii-tp1_01-application\app\src\task_i2c_interface.c
...\sotri_workspace\sotrii-tp1_01-application\app\src\task_i2c_interface.h
...\sotri_workspace\sotrii-tp1_01-application\app\src\freertos.c
```

Leer y almacenar la respuesta en: [...\sotri_workspace\sotrii-tp1_01-application\sotrii-tp1_01-application.md](#).

Paso 04: Depurar el nuevo [proyecto STM32](#).

Confirmar mediante la depuración, el análisis, las explicaciones e indicaciones de la respuesta de [Gemini Pro](#) del **Paso anterior**.

Paso 05: Almacenar el [proyecto STM32](#): [sotrii-tp1_01-application](#) y el/los archivo/s de la actividad: [sotrii-tp1_2-application.md](#), en el [repositorio](#) creado en [GitHub](#), en el **TP1 – Actividad 01 – Paso 01**.

Paso 06: Diseñar, Implementar y Usar [Device Driver I2C](#) de [FreeRTOS](#). con las siguientes características:

Estructura de datos que representen al dispositivo, incluyendo: [device_id](#), [device_queue](#), etc.

Funciones de Interfaz: [open\(\)](#), [release\(\)](#), [write\(\)](#), [read\(\)](#), [ioctl\(\)](#), etc.

Patrón de diseño: [Synchronous](#), [Asynchronous](#), [Latest-Input-Only](#)

Gestión del periférico: [Polling](#), [Interrupt](#), [DMA](#)

Acceso al periférico: [API STM32-F4 HAL](#)

Dos funciones de dispositivo para instanciar tareas del tipo: [Gatekeeper](#) (una para transmitir y una para recibir), que reciben como parámetro referencia al canal [I2C](#) del MCU sobre el que el [Device Driver](#) opera.

Almacenamiento de datos: [Queue](#), [Queue with Input and Output Data Spooler](#) (allocation: [Static](#), [Dynamic](#))

Compilar/depurar/observar comportamiento/**Medir WCET** de las Funciones de Interfaz del driver, **asentar** lo observado en el archivo [sotrii-tp1_01-application.md](#).

Paso 07: Almacenar el [proyecto STM32](#): [sotrii-tp1_01-application](#) y el/los archivo/s de la actividad: [sotrii-tp1_01-application.md](#), en el [repositorio](#) creado en [GitHub](#), en el **TP1 – Actividad 01 – Paso 01**.

TP1 – Actividad 02 – Device Driver UART de FreeRTOS

Objetivos

Diseñar, Implementar y Usar **Device Driver** de **FreeRTOS**

Estructura de datos que representen al dispositivo, incluyendo: **device_id**, **device_queue**, etc.

Funciones de Interfaz: **open()**, **release()**, **write()**, **read()**, **ioctl()**, etc.

Patrones de diseño: **Synchronous**, **Asynchronous**, **Latest Input Only**

Gestión del periférico: **Polling**, **Interrupt**, **DMA**

Acceso al periférico: **API STM32-F4 HAL**

Tareas del tipo: **Gatekeeper**

Almacenamiento de datos: **Queue**, **Queue** with **Input and Output Data Spooler** (allocation: **Static**, **Dynamic**)

Usar **Tareas** de **FreeRTOS**

Diseñar, Implementar, Crear, Eliminar **Tareas** (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias, parámetros, cambio de prioridades, procesamiento periódico, IDLE)

Usar **Comunicación** y **Sincronización** de **Tareas** e **Interrupciones** de **FreeRTOS**

Diseñar, Implementar, Comunicar, Sincronizar **Tareas** entre sí y **Tareas** con **Interrupciones**, mediante **Cola** (bloqueantes o no bloqueantes) o **Semáforo binario** y/o **contador** (bloqueantes o no bloqueantes)

Usar **Sección Crítica** de **FreeRTOS**

Diseñar, Implementar, Proteger **Sección Crítica**, mediante **taskENTER_CRITICAL()/taskEXIT_CRITICAL()**, **vTaskSuspendAll()/xTaskResumeAll()** y **Semáforo Mutex** (Take/Give)

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/Explicar código fuente

Paso 01: Generar un nuevo **proyecto STM32**: **descargar** el archivo [sotrii-tp1_02-application.zip](#) e **importar** el **proyecto STM32** que contiene: **sotrii-tp1_02-application**.

Compilar el **proyecto STM32** (**STM32CubeIDE**) importado: **sotrii-tp1_02-application**.

Paso 02: Asentar los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **sotrii-tp1_02-application.md**.

Paso 03: Consultar a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

Ingresar a **Gemini** => <https://gemini.google.com/app>, en **Preguntar a Gemini** =>

Copiar y Pegar la siguiente línea de **texto**:

Analizar y explicar (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: [app.c](#), [app_it.c](#), [task_sender.c](#), [task_receiver.c](#), [task_uart.c](#), [task_uart_interface.c](#) y [freertos.c](#).

Agregar los siguientes archivos (click en **+** Agregar archivos => **Subir archivos**):

```
...|sotri_workspace\sotrii-tp1_02-application\app\src\app.c
...|sotri_workspace\sotrii-tp1_02-application\app\src\app_it.c
...|sotri_workspace\sotrii-tp1_02-application\app\src\task_sender.c
...|sotri_workspace\sotrii-tp1_02-application\app\src\task_receiver.c
...|sotri_workspace\sotrii-tp1_02-application\app\src\task_uart.c
...|sotri_workspace\sotrii-tp1_02-application\app\src\task_uart_interface.c
...|sotri_workspace\sotrii-tp1_02-application\app\incl\task_uart_interface.h
...|sotri_workspace\sotrii-tp1_02-application\app\src\freertos.c
```

Leer y almacenar la respuesta en: ...**sotri_workspace\sotrii-tp1_02-application\sotrii-tp1_02-application.md**.

Paso 04: Depurar el nuevo proyecto STM32.

Confirmar mediante la depuración, el análisis, las explicaciones e indicaciones de la respuesta de **Gemini Pro** del **Paso anterior**.

Paso 05: Almacenar el [proyecto STM32: sotriip-tp1_02-application](#) y el/los archivo/s de la actividad: [sotriip-tp1_3-application.md](#), en el [repositorio](#) creado en [GitHub](#), en el **TP1 – Actividad 01 – Paso 01**.

Paso 06: Diseñar, Implementar y Usar Device Driver UART de FreeRTOS. con las siguientes características:

Estructura de datos que representen al dispositivo, incluyendo: **device_id**, **device_queue**, etc.

Funciones de Interfaz: **open()**, **release()**, **write()**, **read()**, **ioctl()**, etc.

Patrón de diseño: **Synchronous**, **Asynchronous**, **Latest Input Only**

Gestión del periférico: **Polling**, **Interrupt**, **DMA**

Accesso al periférico: **API STM32-F4 HAL**

Dos funciones de dispositivo para instanciar tareas del tipo: **Gatekeeper** (una para transmitir y una para recibir), que reciben como parámetro referencia al canal **UART** del MCU sobre el que el **Device Driver** opera.

Almacenamiento de datos: **Queue**, **Queue with Input and Output Data Spooler** (allocation: **Static**, **Dynamic**)

Compilar/depurar/observar comportamiento/**Medir WCET** de las Funciones de Interfaz del driver, **asentar** lo observado en el archivo **sotrii-tp1 02-application.md**.

Paso 07: Almacenar el [proyecto STM32: sotrii-tp1_02-application](#) y el/los archivo/s de la actividad: [sotrii-tp1_02-application.md](#), en el [repositorio](#) creado en [GitHub](#), en el **TP1 – Actividad 01 – Paso 01**.

TP1 – Actividad 03 – Device Driver ADC de FreeRTOS

Objetivos

Diseñar, Implementar y Usar **Device Driver** de **FreeRTOS**

Estructura de datos que representen al dispositivo, incluyendo: **device_id**, **device_queue**, etc.

Funciones de Interfaz: **open()**, **release()**, **write()**, **read()**, **ioctl()**, etc.

Patrones de diseño: **Synchronous**, **Asynchronous**, **Latest Input Only**

Gestión del periférico: **Polling**, **Interrupt**, **DMA**

Acceso al periférico: **API STM32-F4 HAL**

Tareas del tipo: **Gatekeeper**

Almacenamiento de datos: **Queue**, **Queue** with **Input and Output Data Spooler** (allocation: **Static**, **Dynamic**)

Usar **Tareas** de **FreeRTOS**

Diseñar, Implementar, Crear, Eliminar **Tareas** (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias, parámetros, cambio de prioridades, procesamiento periódico, IDLE)

Usar **Comunicación** y **Sincronización** de **Tareas** e **Interrupciones** de **FreeRTOS**

Diseñar, Implementar, Comunicar, Sincronizar **Tareas** entre sí y **Tareas** con **Interrupciones**, mediante **Cola** (bloqueantes o no bloqueantes) o **Semáforo binario** y/o **contador** (bloqueantes o no bloqueantes)

Usar **Sección Crítica** de **FreeRTOS**

Diseñar, Implementar, Proteger **Sección Crítica**, mediante **taskENTER_CRITICAL()/taskEXIT_CRITICAL()**, **vTaskSuspendAll()/xTaskResumeAll()** y **Semáforo Mutex** (Take/Give)

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/Explicar código fuente

Paso 01: Generar un nuevo **proyecto STM32**: **descargar** el archivo [sotrii-tp1_03-application.zip](#) e **importar** el **proyecto STM32** que contiene: **sotrii-tp1_03-application**.

Compilar el **proyecto STM32** (**STM32CubeIDE**) importado: **sotrii-tp1_03-application**.

Paso 02: Asentar los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **sotrii-tp1_03-application.md**.

Paso 03: Consultar a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

Ingresar a **Gemini** => <https://gemini.google.com/app>, en **Preguntar a Gemini** =>

Copiar y Pegar la siguiente línea de **texto**:

Analizar y explicar (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: [app.c](#), [app_it.c](#), [task_receiver.c](#), [task_adc.c](#), [task_adc_interface.c](#), [task_adc_interface.h](#) y [freertos.c](#).

Agregar los siguientes archivos (click en  Agregar archivos => **Subir archivos**):

```
...\sotri_workspace\sotrii-tp1_03-application\app\src\app.c
...\sotri_workspace\sotrii-tp1_03-application\app\src\app_it.c
...\sotri_workspace\sotrii-tp1_03-application\app\src\task_receiver.c
...\sotri_workspace\sotrii-tp1_03-application\app\src\task_adc.c
...\sotri_workspace\sotrii-tp1_03-application\app\src\task_adc_interface.c
...\sotri_workspace\sotrii-tp1_03-application\app\src\task_adc_interface.h
...\sotri_workspace\sotrii-tp1_03-application\app\src\freertos.c
```

Leer y almacenar la respuesta en: [...\sotri_workspace\sotrii-tp1_03-application\sotrii-tp1_03-application.md](#).

Paso 04: Depurar el nuevo [proyecto STM32](#).

Confirmar mediante la depuración, el análisis, las explicaciones e indicaciones de la respuesta de [Gemini Pro](#) del **Paso anterior**.

Paso 05: Almacenar el [proyecto STM32: sotrii-tp1_03-application](#) y el/los archivo/s de la actividad: [sotrii-tp1_4-application.md](#), en el [repositorio](#) creado en [GitHub](#), en el **TP1 – Actividad 01 – Paso 01**.

Paso 06: Diseñar, Implementar y Usar [Device Driver ADC](#) de [FreeRTOS](#), con las siguientes características:

Estructura de datos que representen al dispositivo, incluyendo: [device_id](#), [device_queue](#), etc.

Funciones de Interfaz: [open\(\)](#), [release\(\)](#), [write\(\)](#), [read\(\)](#), [ioctl\(\)](#), etc.

Patrón de diseño: [Synchronous](#), [Asynchronous](#), [Latest Input Only](#)

Gestión del periférico: [Polling](#), [Interrupt](#), [DMA](#)

Acceso al periférico: [API STM32-F4 HAL](#)

Una función de dispositivo para instanciar tareas del tipo: [Gatekeeper](#) (para recibir), que reciben como parámetro referencia al canal [ADC](#) del MCU sobre el que el [Device Driver](#) opera.

Almacenamiento de datos: [Queue](#), [Queue with Input and Output Data Spooler](#) (allocation: [Static](#), [Dynamic](#))

Compilar/depurar/observar comportamiento/**Medir WCET** de las Funciones de Interfaz del driver, **asentar** lo observado en el archivo [sotrii-tp1_03-application.md](#).

Paso 07: Almacenar el [proyecto STM32: sotrii-tp1_03-application](#) y el/los archivo/s de la actividad: [sotrii-tp1_03-application.md](#), en el [repositorio](#) creado en [GitHub](#), en el **TP1 – Actividad 01 – Paso 01**.

GitHub: Presionar: **Commit changes ...** (**Guardar** los cambios)

Tildar: **Create a new branch** for this commit and start a pull request

Presionar: **Propose changes**

Presionar: **Create pull request**, tipear el nombre del **branch** y **agregar** a los [docentes](#) de su curso, como **Reviewers** (para invitarlos a **revisar** sus **entregas** mediante un **pull request**)

Ver **cómo hacer el pull request** en este video: [Trabajo colaborativo con Github](#)

En la descripción del video se detalla el contenido (ver en [14:23](#) la sección "**CÓMO SOLICITAR REVISIONES**")
IMPORTANTE: Se recomienda que primero vean "[19:25 Solución a un problema muy habitual](#)" y luego "[19:02 Create pull request y Review requested](#)"